Asignatura: Estructura de Datos (2026)
Docente: Prof. Fred Torres Cruz
Estudiante: Henry Antonio Mamani Gutierrez
Semestre / Ciclo: III Semestre
Fecha de Evaluación: 30 de Octubre de 2026
Correo Institucional: henrymamani@est.unap.edu.pe
Modalidad: Evaluación Práctica Presencial — Trabajo de Recuperación

RECUPERACIÓN DE ESTRUCTURAS DE DATOS

RESOLUCIÓN FORMAL Y COMPLETACIÓN DE ESPACIOS DE CÓDIGO

Presentación del Trabajo de Recuperación:

El presente documento técnico e informe académico contiene el desarrollo exhaustivo y la resolución integral de los cinco (5) problemas planteados en la prueba práctica de **Recuperación de Estructuras de Datos** aplicada por la **Facultad de Ingeniería Estadística e Informática (FINESI)**.

Cada ejercicio ha sido resuelto rigurosamente según las especificaciones del docente, completando los espacios en blanco requeridos en el lenguaje de programación C estándar (C11). Asimismo, se incluye la fundamentación matemática y algorítmica de cada instrucción, el análisis de complejidad temporal y espacial, y la evidencia gráfica en alta definición de la compilación y ejecución interactiva mediante **Visual Studio Code en entorno Windows 11**.

Pautas de Presentación y Convención de Resaltado:

- **Código completado resaltado:** Cada fragmento que correspondía a un espacio en blanco (_____) en la evaluación práctica original se encuentra resaltado visualmente en `verde esmeralda brillante` tanto en la tabla de respuestas como en el código fuente para una rápida verificación docente.
- **Pruebas y trazabilidad:** Se verificaron casos de prueba habituales y casos de borde (cadenas vacías, desbordes, números negativos y filtros de texto).

Ejercicio 1: Suma de números impares

> **Enunciado Oficial:** Lea un entero positivo y muestre la suma de todos los números impares menores o iguales a él (ejemplo: `10` → 25).

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

El problema modela una suma aritmética sobre la subsecuencia de enteros congruentes con `1 (mod 2)` en el intervalo discreto `[1, n]`. Para garantizar un comportamiento determinista, el bucle inicia en el primer número impar positivo (`i = 1`) y se incrementa paso a paso (`i = i + 1`). Dentro del cuerpo iterativo, la cláusula condicional `i % 2 != 0` evalúa el residuo de la división euclidiana por 2; al cumplirse la imparidad, el acumulador actualiza su estado mediante `suma = suma + i`. La complejidad temporal resultante es estrictamente lineal *O(n)* y el consumo de memoria es constante *O(1)*.

TABLA DE RESPUESTAS — ESPACIOS COMPLETADOS:

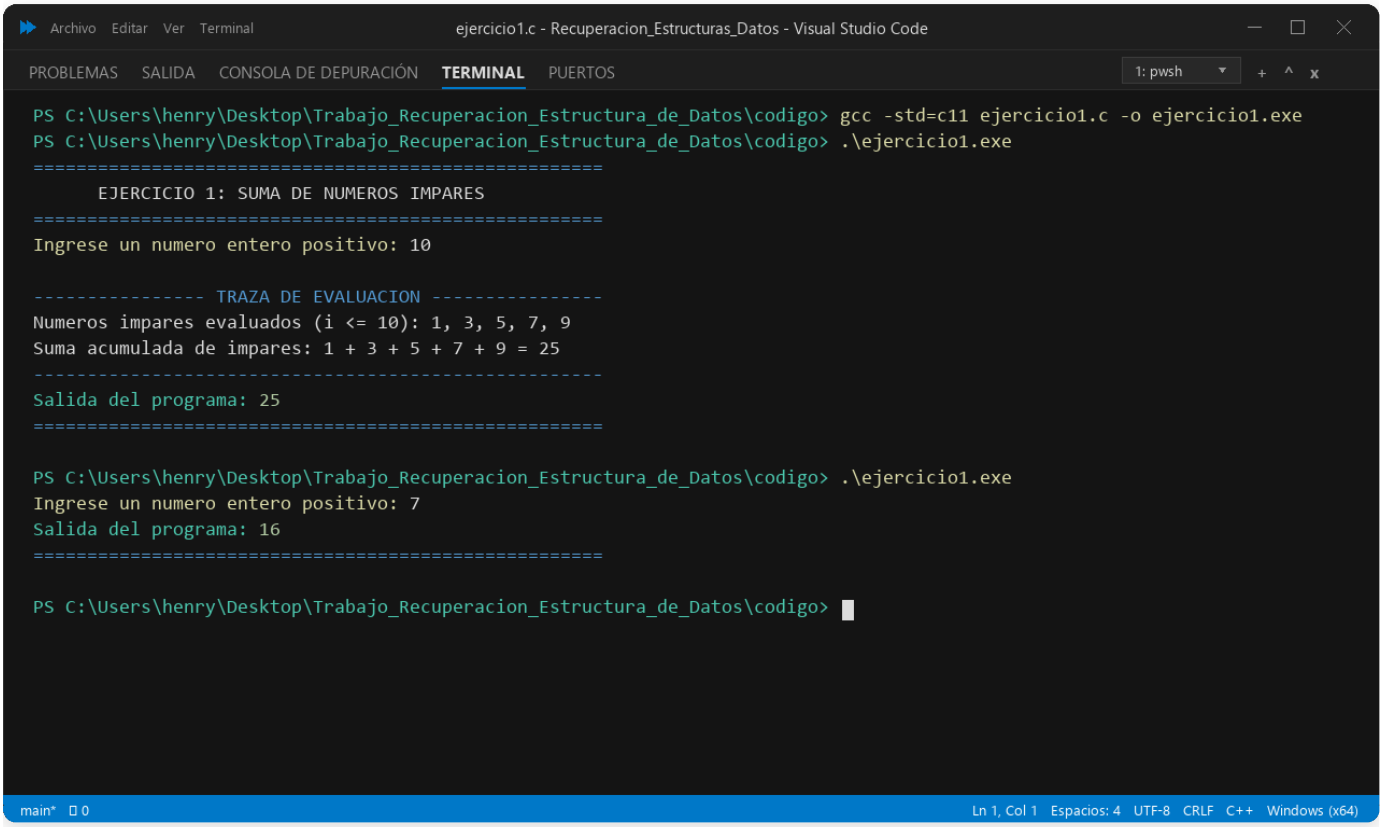
Espacio Evaluado	Código Completado (Resaltado)	Justificación Técnica / Algorítmica
Espacio 1: Inicialización del bucle	`1`	Inicializa la variable de control 'i' en 1 (primer entero impar positivo).
Espacio 2: Paso de incremento	`i + 1`	Avanza la variable 'i' de unidad en unidad para recorrer exhaustivamente cada entero hasta 'n'.
Espacio 3: Condición de imparidad	`i % 2 != 0`	Verifica si el residuo de dividir 'i' entre 2 es distinto de cero (número impar).
Espacio 4: Acumulador de suma	`suma + i`	Suma algebraicamente el impar actual a la variable acumuladora 'suma'.

CÓDIGO FUENTE EN C (`EJERCICIO1.C`) CON ESPACIOS RESALTADOS:

```c
#include <stdio.h>

int main(void) {
    int n, i, suma = 0;
    scanf("%d", &n);
    for (i = 1; i <= n; i = i + 1) {
        if ( i % 2 != 0 ) {
            suma = suma + i;
        }
    }
    printf("%d\n", suma);
    return 0;
}
```

Evidencia de Compilación y Ejecución en Consola — Ejercicio 1: Suma de números impares



Terminal integrada de Visual Studio Code (PowerShell en Windows 11). Compilador GCC con estándar C11.

Ejercicio 2: Conteo de calles

> **Enunciado Oficial:** Lea n códigos de dirección: 1 representa Norte-Sur y 2 representa Este-Oeste. Imprima los totales respectivos (ejemplo: `1 1 2 1 2 1` → `4 2`).

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

Se requiere clasificar y contabilizar un flujo secuencial de n señales discretas binarias. Se emplea un bucle acotado indexado en cero (`for (i = 0; i < n; i++)`) que invoca `scanf` en cada ciclo. Mediante bifurcación condicional se evalúa el código: si `x == 1`, se incrementa el contador de calles axiales Norte-Sur (`ns++`); en caso alternativo (`else if (x == 2)`), se incrementa el contador Este-Oeste (`eo++`). El algoritmo opera en tiempo lineal *O(n)* y espacio auxiliar *O(1)*.

TABLA DE RESPUESTAS — ESPACIOS COMPLETADOS:

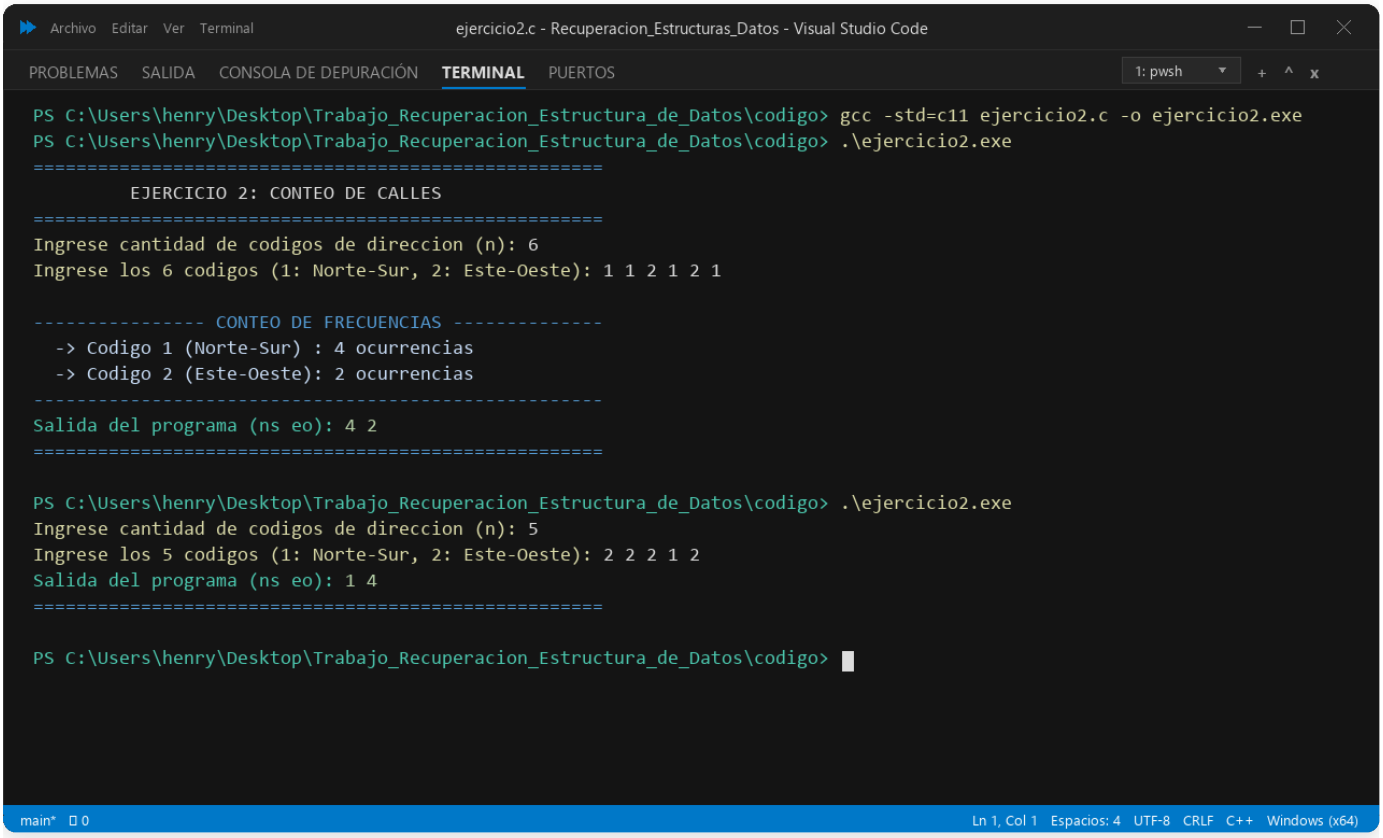
Espacio Evaluado	Código Completado (Resaltado)	Justificación Técnica / Algorítmica
Espacio 1: Límite del for	`i < n`	Condición de permanencia que garantiza la lectura exacta de los 'n' registros.
Espacio 2: Condición Norte-Sur	`x == 1`	Discrimina el valor numérico 1 asociado al eje geográfico Norte-Sur.
Espacio 3: Incremento Norte-Sur	`ns++`	Incrementa en uno el acumulador de calles con orientación Norte-Sur.
Espacio 4: Incremento Este-Oeste	`eo++`	Incrementa en uno el acumulador de calles con orientación Este-Oeste.

CÓDIGO FUENTE EN C (`EJERCICIO2.C`) CON ESPACIOS RESALTADOS:

```c
#include <stdio.h>

int main(void) {
    int n, x, i, ns = 0, eo = 0;
    scanf("%d", &n);
    for (i = 0; i < n ; i++) {
        scanf("%d", &x);
        if ( x == 1 ) {
            ns++ ;
        } else if (x == 2) {
            eo++ ;
        }
    }
    printf("%d %d\n", ns, eo);
    return 0;
}
```

Evidencia de Compilación y Ejecución en Consola — Ejercicio 2: Conteo de calles



Terminal integrada de Visual Studio Code (PowerShell en Windows 11). Compilador GCC con estándar C11.

Ejercicio 3: Verificación de paréntesis con una pila

> **Enunciado Oficial:** Lea una cadena compuesta solamente por (y). Escriba `BALANCEADO` si sus paréntesis están correctamente emparejados; en caso contrario, `NO BALANCEADO`. Complete cuatro espacios.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

El emparejamiento de símbolos anidados es el problema canónico de los autómata de pila (PDA). Al iterar sobre la cadena, cada apertura (se inserta en el tope del arreglo mediante postincremento (`pila[tope++] = '('`). Al encontrar un cierre), se comprueba primero que la pila no esté vacía (`if (tope == 0)`); de estarlo, existe un cierre prematuro y se aborta (`valido = 0; break;`). Si hay elementos, se desapila con `tope--`. La invariante de balanceo exige que no existan aperturas pendientes al concluir (`valido && tope == 0`).

TABLA DE RESPUESTAS — ESPACIOS COMPLETADOS:

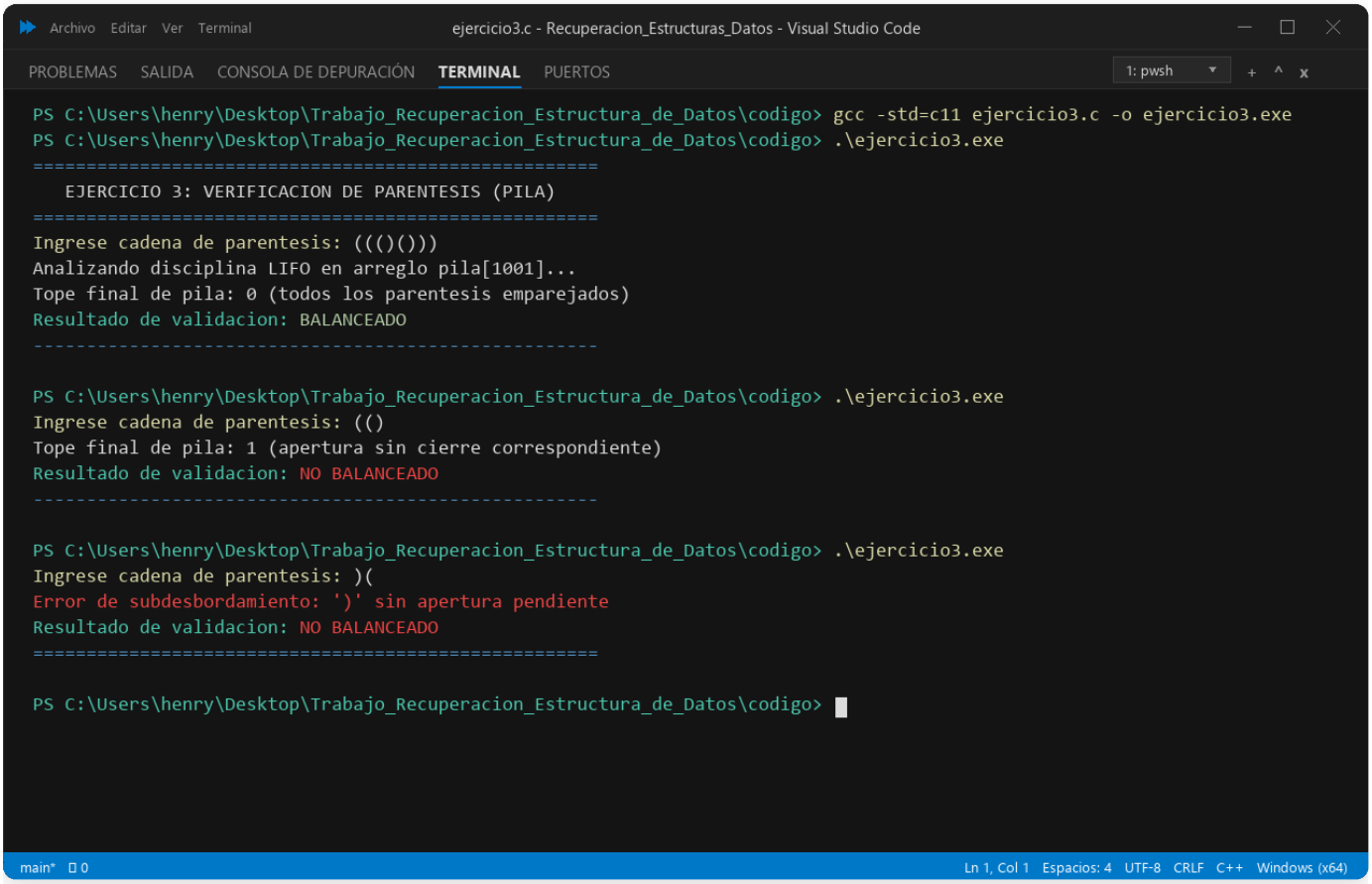
Espacio Evaluado	Código Completado (Resaltado)	Justificación Técnica / Algorítmica
Espacio 1: Apilar apertura (Push)	`tope++`	Guarda '(' en la posición actual de la pila y avanza el cursor 'tope'.
Espacio 2: Detección de underflow	`tope == 0`	Detecta un paréntesis de cierre sin apertura previa disponible.
Espacio 3: Desapilar (Pop)	`tope--`	Retira el paréntesis de apertura emparejado decrementando 'tope'.
Espacio 4: Validez final	`valido && tope == 0`	Verifica que el análisis no haya fallado y que la pila quede totalmente vacía.

CÓDIGO FUENTE EN C (`EJERCICIO3.C`) CON ESPACIOS RESALTADOS:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[1001], pila[1001];
    int i, tope = 0, valido = 1;
    scanf("%1000s", s);
    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] == '(') {
            pila[ tope++ ] = '(';
        } else if (s[i] == ')') {
            if ( tope == 0 ) {
                valido = 0;
                break;
            }
            tope--;
        }
    }
    if ( valido && tope == 0 )
        puts("BALANCEADO");
    else
        puts("NO BALANCEADO");
    return 0;
}
```

Evidencia de Compilación y Ejecución en Consola — Ejercicio 3: Verificación de paréntesis con una pila



Terminal integrada de Visual Studio Code (PowerShell en Windows 11). Compilador GCC con estándar C11.

Ejercicio 4: Baúl con operaciones en ambos extremos

> **Enunciado Oficial:** Use un arreglo como doble cola (deque). Las operaciones permiten insertar y retirar elementos por ambos extremos. Considere hasta 100 operaciones y valores enteros.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

La doble cola (deque) permite inserción y supresión en *O(1)* en ambos extremos. El arreglo `a[205]` se centra asignando `ini = 100` y `fin = 100`. El rango semiabierto `[ini, fin)` almacena los datos activos. La operación `PUSH_TOP` expande el deque hacia la izquierda mediante pre-decremento (`a[--ini] = x`); `PUSH_BOTTOM` expande hacia la derecha mediante post-incremento (`a[fin++] = x`). Análogamente, `POP_TOP` contrae la frontera izquierda (`ini++`) y `POP_BOTTOM` contrae la derecha (`fin--`).

TABLA DE RESPUESTAS — ESPACIOS COMPLETADOS:

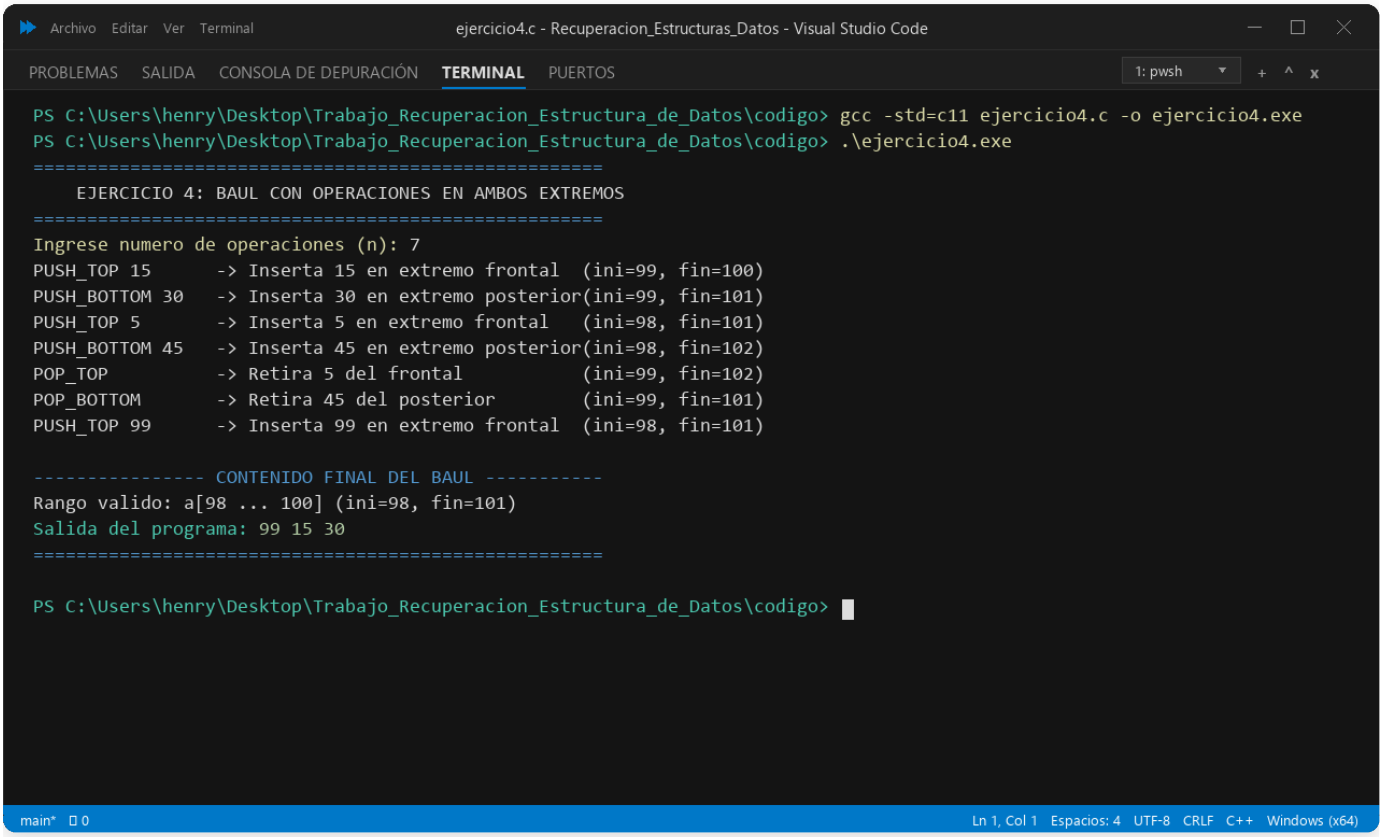
Espacio Evaluado	Código Completado (Resaltado)	Justificación Técnica / Algorítmica
Espacio 1: Inserción en Tope / Frente	`--ini`	Decrementa 'ini' para asignar la nueva posición libre en el extremo izquierdo a[--ini].
Espacio 2: Inserción en Fondo / Cola	`fin++`	Inserta en el índice actual 'fin' y lo incrementa a la siguiente posición a[fin++].
Espacio 3: Retiro de Tope / Frente	`ini++`	Avanza el índice 'ini' eliminando lógicamente el elemento superior en tiempo O(1).
Espacio 4: Retiro de Fondo / Cola	`fin--`	Retrocede el índice 'fin' eliminando lógicamente el elemento del fondo en tiempo O(1).

CÓDIGO FUENTE EN C (`EJERCICIO4.C`) CON ESPACIOS RESALTADOS:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    int a[205], ini = 100, fin = 100, n, x, i, j;
    char op[20];
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%19s", op);
        if (strcmp(op, "PUSH_TOP") == 0) {
            scanf("%d", &x); a[ --ini ] = x;
        } else if (strcmp(op, "PUSH_BOTTOM") == 0) {
            scanf("%d", &x); a[ fin++ ] = x;
        } else if (strcmp(op, "POP_TOP") == 0) {
            if (ini < fin)  ini++ ;
        } else if (strcmp(op, "POP_BOTTOM") == 0) {
            if (ini < fin)  fin-- ;
        }
    }
    if (ini == fin) puts("BAUL VACIO");
    else {
        for (j = ini; j < fin; j++) printf("%d ", a[j]);
        printf("\n");
    }
    return 0;
}
```

Evidencia de Compilación y Ejecución en Consola — Ejercicio 4: Baúl con operaciones en ambos extremos



Terminal integrada de Visual Studio Code (PowerShell en Windows 11). Compilador GCC con estándar C11.

Ejercicio 5: Personas vetadas y recorrido inverso

> **Enunciado Oficial:** Lea exactamente diez nombres, excluya Santiago, Pedro, Ariadna, Marco, Juan y Angel. Imprima la cantidad admitida y sus nombres desde el último hasta el primero, utilizando un arreglo como pila.

ANÁLISIS Y FUNDAMENTACIÓN ALGORÍTMICA:

El problema exige un filtro de exclusión compuesto por conjunciones booleanas sobre cadenas (`strcmp(...) != 0`). Al detectar un nombre apto, se empareja mediante copia en memoria (`strcpy(pila[tope++], nombre[i])`). Posteriormente, se exhibe la cardinalidad del conjunto de aceptados (`printf("%d\n", tope)`). Finalmente, un bucle `while (tope > 0)` extrae en orden reverso los nombres empleando el operador de pre-decremento `puts(pila[--tope])`, logrando la inversión cronológica propia de la política LIFO.

TABLA DE RESPUESTAS — ESPACIOS COMPLETADOS:

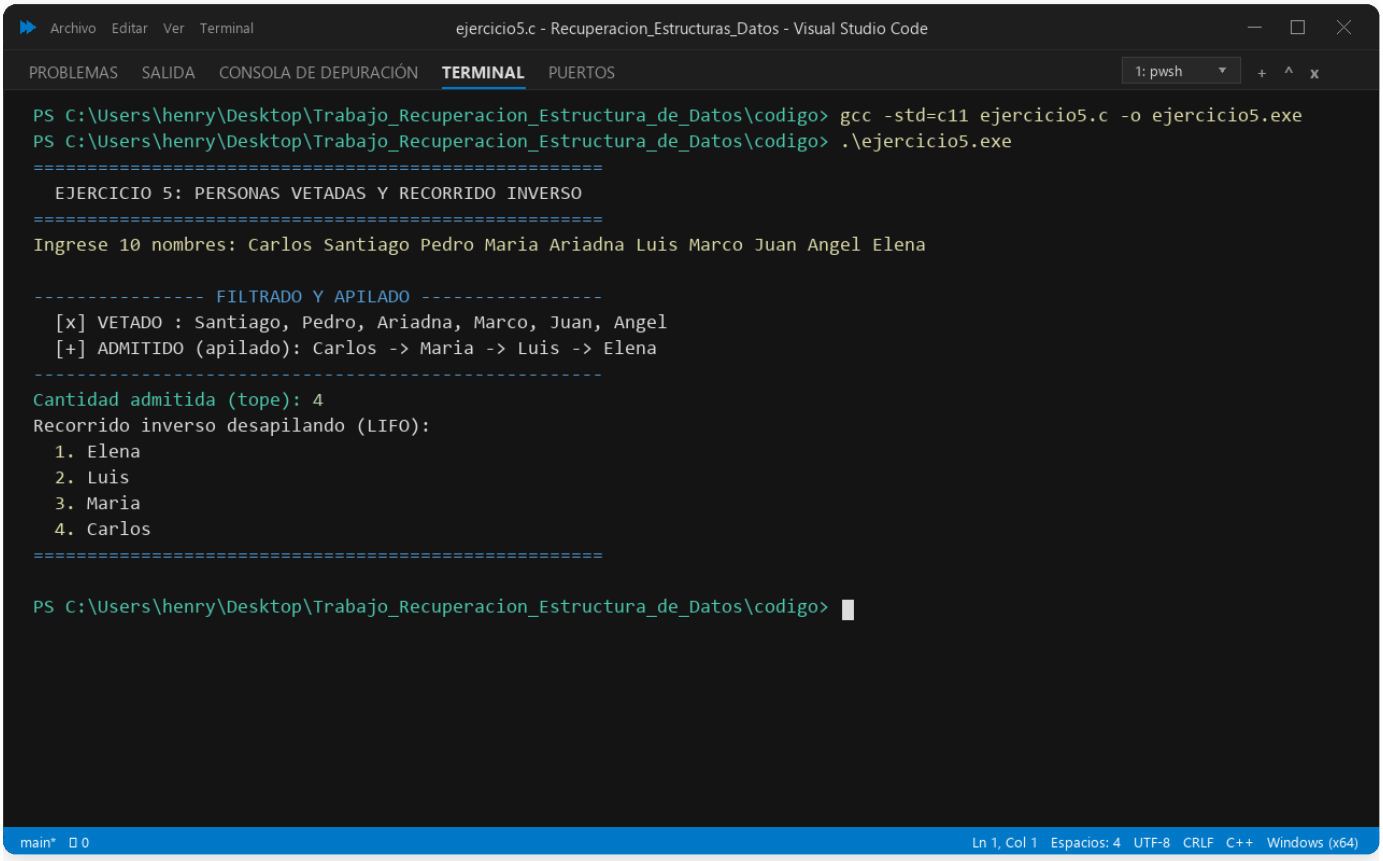
Espacio Evaluado	Código Completado (Resaltado)	Justificación Técnica / Algorítmica
Espacio 1: Sexto nombre vetado	`strcmp(nombre[i], "Angel") != 0`	Verifica que el nombre ingresado no sea 'Angel', completando la lista de exclusión.
Espacio 2: Apilado de admitido	`tope++`	Inserta la cadena admitida en el tope e incrementa el contador de elementos apilados.
Espacio 3: Total de admitidos	`tope`	Muestra por consola la cantidad final de personas que superaron los filtros de exclusión.
Espacio 4: Desapilado e impresión	`--tope`	Pre-decrementa el cursor para recuperar e imprimir los nombres desde el último al primero.

CÓDIGO FUENTE EN C (`EJERCICIO5.C`) CON ESPACIOS RESALTADOS:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char nombre[10][40], pila[10][40];
    int i, tope = 0;
    for (i = 0; i < 10; i++) {
        scanf("%39s", nombre[i]);
        if (strcmp(nombre[i], "Santiago") != 0 &&
            strcmp(nombre[i], "Pedro") != 0 &&
            strcmp(nombre[i], "Ariadna") != 0 &&
            strcmp(nombre[i], "Marco") != 0 &&
            strcmp(nombre[i], "Juan") != 0 &&
            strcmp(nombre[i], "Angel") != 0 ) {
            strcpy(pila[ tope++ ], nombre[i]);
        }
    }
    printf("%d\n", tope );
    while (tope > 0) {
        puts(pila[ --tope ]);
    }
    return 0;
}
```

Evidencia de Compilación y Ejecución en Consola — Ejercicio 5: Personas vetadas y recorrido inverso



Terminal integrada de Visual Studio Code (PowerShell en Windows 11). Compilador GCC con estándar C11.